

Aufgabe 1.1: Syntaxhighlighting mit dem AntlrTokenCollector

Überlegen Sie, ob und wie in diesem Fall noch Konflikte zwischen Regionen entstehen können und wie die beiden Hook-Methoden normalize und resolveConflicts aus der Oberklasse AntlrTokenCollector in diesem Fall aussehen müssen. Überschreiben Sie die Methoden entsprechend in AntlrTokenCollector (falls notwendig).:

- Konflikte können hier praktisch nicht mehr entstehen, weil ANTLR den Eingabetext eindeutig in Tokens zerlegt. Jeder relevante Token liefert höchstens eine HighlightRegion. Die Annotation besteht zwar aus zwei Tokens, aber der Identifier wird nicht nochmal separat markiert. Deshalb ist keine echte Konfliktauflösung nötig.
- normalize und resolveConflicts müssen nicht überschrieben werden, weil der ANTLR-Lexer bereits geordnete, nicht überlappende Tokens erzeugt und dadurch keine Highlight-Konflikte entstehen.

Vergleichen Sie das so erzeugte Syntaxhighlighting mit den Varianten von Blatt 04. Wo gibt es Unterschiede, was sind die Gründe dafür? Welche der Varianten ist aufwändiger in der Implementierung?

Beim **ANTLR-Collector** wird das Syntaxhighlighting über den generierten Lexer erzeugt. Dadurch erkennt der Lexer zuerst die Struktur des Quelltexts als Tokens und danach werden diese Tokens eingefärbt.

Im Vergleich zu Blatt 04 gibt es diese Unterschiede:

Bei der **Regex-Variante** wurden Muster direkt im Text gesucht, z.B. für Kommentare, Strings, Keywords oder Annotationen. Dadurch konnten leichter Überschneidungen entstehen, etwa wenn ein Keyword innerhalb eines Strings oder Kommentars vorkommt. Deshalb waren Normalisierung und Konfliktauflösung wichtiger.

Bei der **ANTLR-Variante** zerlegt der Lexer den Text vorher eindeutig in Tokens. Ein Kommentar ist dann ein Kommentar-Token, ein String ist ein String-Token, ein Keyword ist ein Keyword-Token. Dadurch gibt es praktisch keine überlappenden Regionen mehr.

Ein sichtbarer Unterschied ist z.B. bei Annotationen wie:

@Over-ride

Die alte Regex-Variante kann das komplett markieren, weil das Regex "- " erlaubt hat. Der ANTLR-Lexer zerlegt es aber in @, Over, -, ride, weil "- " kein gültiger Teil eines Java-Identifiers ist. Deshalb wird nur @Over als Annotation erkannt. Der Grund ist: ANTLR folgt der Grammatik, Regex folgt nur dem selbst geschriebenen Suchmuster.

Die **ANTLR-Variante ist fachlich sauberer**, weil sie sich an der Grammatik orientiert und syntaktische Einheiten erkennt. Die **Regex-Variante ist einfacher und schneller geschrieben**, aber fehleranfälliger, weil man Reihenfolge, Überschneidungen und Sonderfälle selbst behandeln muss.

Insgesamt ist die **ANTLR-Grundstruktur aufwändiger**, weil Grammatik, generierter Lexer und Token-Typen verstanden werden müssen. Die eigentliche Implementierung im Collector ist danach aber oft einfacher und robuster, weil der Lexer die schwere Erkennungsarbeit übernimmt.

Aufgabe 1.2: Visitoren mit ANTLR (Pretty Printing):

Schritt 3: Benutzung und Demonstration

- Fragen Sie auf der Konsole nach der gewünschten Anzahl von Leerzeichen pro Einrückstufe (z.B. 2, 4 oder 8).
- Übergeben Sie diese Zahl an den Konstruktor `PrettyPrinterVisitor(int indentWidth)` und verwenden Sie sie zum Einrücken.
- Geben Sie den von Ihrem Pretty-Printer erzeugten String z.B. auf der Konsole aus.
- Demonstrieren und dokumentieren Sie Ihre Lösung an mehreren Beispielen mit unterschiedlicher Komplexität, z.B.:
 - einfache Klasse mit einem Feld und einer Methode,
 - Methoden mit `if/else` und `while`,
 - verschachtelte Blöcke.

Code in der Main-Methode:

```
import highlighting.antlr.*;
import highlighting.regex.*;
import org.antlr.v4.runtime.CharStreams;
import org.antlr.v4.runtime.CommonTokenStream;
```

```
public static void main(String[] args) { new *
    int indentWidth = 4;

    String example1 =
        """
        |package demo.simple; import java.util.List; public class SimpleExample { private String name; public String getName() { return name; } }
        """;

    String example2 =
        """
        |public class ControlExample { private String value; public String check(String input) { if (input == null) { return "empty"; } else { while (value != null) { value = null; } return input; } } }
        """;

    String example3 =
        """
        |public class NestedExample { public String run(String text) { { String result = text; { if (result == null) { result = "fallback"; } else { { result = "ok"; } } } return result; } } }
        """;

    String[] examples = {example1, example2, example3};

    for (int i = 0; i < examples.length; i++) {
        var input = CharStreams.fromString(examples[i]);
        var lexer = new MiniJavaLexer(input);
        var tokens = new CommonTokenStream(lexer);
        var parser = new MiniJavaParser(tokens);

        MiniJavaParser.CompilationUnitContext tree = parser.compilationUnit();

        var visitor = new PrettyPrinterVisitor(indentWidth);
        visitor.visit(tree);

        System.out.println("==== Beispiel " + (i + 1) + " =====");
        System.out.println(visitor.result());
        System.out.println();
    }
}
```

```
11:51:53: Executing ':Main.main()'...
```

```
Starting Gradle Daemon...
```

```
Gradle Daemon started in 10 s 958 ms
```

```
> Task :generateGrammarSource UP-TO-DATE
```

```
> Task :compileJava
```

```
> Task :processResources NO-SOURCE
```

```
> Task :classes
```

```
> Task :Main.main()
```

```
===== Beispiel 1 =====
```

```
package demo.simple;
```

```
import java.util.List;
```

```
public class SimpleExample {  
    private String name;  
    public String getName() {  
        return name;  
    }  
}
```

```
===== Beispiel 2 =====
```

```
public class ControlExample {  
    private String value;  
    public String check(String input) {  
        if (input==null) {  
            return "empty";  
        }  
        else {  
            while (value!=null) {  
                value=null;  
            }  
            return input;  
        }  
    }  
}
```

Ergebnisse:

```
===== Beispiel 3 =====
```

```
public class NestedExample {  
    public String run(String text) {  
        {  
            String result=text; {  
                if (result==null) {  
                    result="fallback";  
                }  
                else {  
                    {  
                        result="ok";  
                    }  
                }  
            }  
        }  
        return result;  
    }  
}
```

```
BUILD SUCCESSFUL in 20s
```

```
3 actionable tasks: 2 executed, 1 up-to-date
```

```
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.5.1/userguide/configuration\_cache\_enabling.html
```

```
11:52:14: Execution finished ':Main.main()'.
```